

Claude Code 记忆 (Memory) 系统设计

记忆语义分类、召回、写入、更新与维护

五道口纳什 & Hermes

2026-07-16

Memory System 记忆 Claude Code

视频作者/频道: 五道口纳什

发布日期: 2026-04-03

视频时长: 21:38

视频链接: <https://www.bilibili.com/video/BV1ZA93BtEKW>

PDF 制作: Bilibili Render PDF (Hermes Agent)

目录

1 引言：为什么讲 Claude Code 的记忆系统

视频开场白 (00:00:00–00:00:33)

五道口纳什： 好亲爱的朋友们，大家中午好！今天我们继续回到 Modern Agent 系列。本来我不是很想讲 Claude Code 或者 Codex 了，但是最近 Claude Code 的源码泄露，加上我一直对 Memory 的设计实现很好奇，这次终于可以详细分析 Claude Code 如何去设计和实现它的 Memory System。

这期视频围绕几个核心话题展开：

- 1. **Memory 的语义分类**——按 user / feedback / project / reference 四类
- 2. **本地持久化**——如何落盘、更新、维护
- 3. **召回与注入**——在 Agent Loop 中如何加载记忆，以及对应的 Wire Layer（网络层）

核心线索

整个记忆系统的设计围绕一条主线：**语言模型驱动**。无论是写入、加载还是注入 Agent Loop，都涉及大量的提示词工程（Prompt Engineering），去避免 Memory 幻觉——就像人的记忆一样，可能会记不清或记错。

2 四维语义分类

视频重点介绍了 Claude Code 的 Auto Memory 按语义的四种闭合分类：

类型	解决的问题	本质
user	这个用户是谁？	个体式背景画像
feedback	我以后该怎么做？	行为约束型规则
project	项目处在什么外部环境？	非代码可推导上下文
reference	去哪找外部信息？	外部系统指针

表 1: 四种记忆类型的语义定位

2.1 User

user 被定义为——关于**用户是谁**的记忆，而不是项目的事实。它改变的是：回答的风格、解释的力度、默认的协作方式。

2.2 Feedback

feedback 不仅记录负反馈（纠正），也记录**正反馈**（认可）。每条 feedback 记忆的结构是：

- 1

正文结构

- 2

Rule / Facts

（规则或事实）
- 3

Why:

（为什么会有这条规则——理解边界条件）

Listing 1: Feedback 记忆的结构要求

关键设计思想：如果没有 Why，模型以后只能机械地套用规则；有了 Why，它才知道边界条件——在什么情况下这条规则适用或不适用。

2.3 Project

project 存的是**不能从代码直接推出的上下文**——比如截止日期、Merge 冻结期、某次重构背后的合规原因、某个事故的业务背景。这些东西即使读完整个 Repo 也未必能推出。

2.4 Reference

reference 不是存事实本身，而是存**去哪找事实**。比如某个 Linear Project 的看板、某个 Slack Channel 存什么信息。

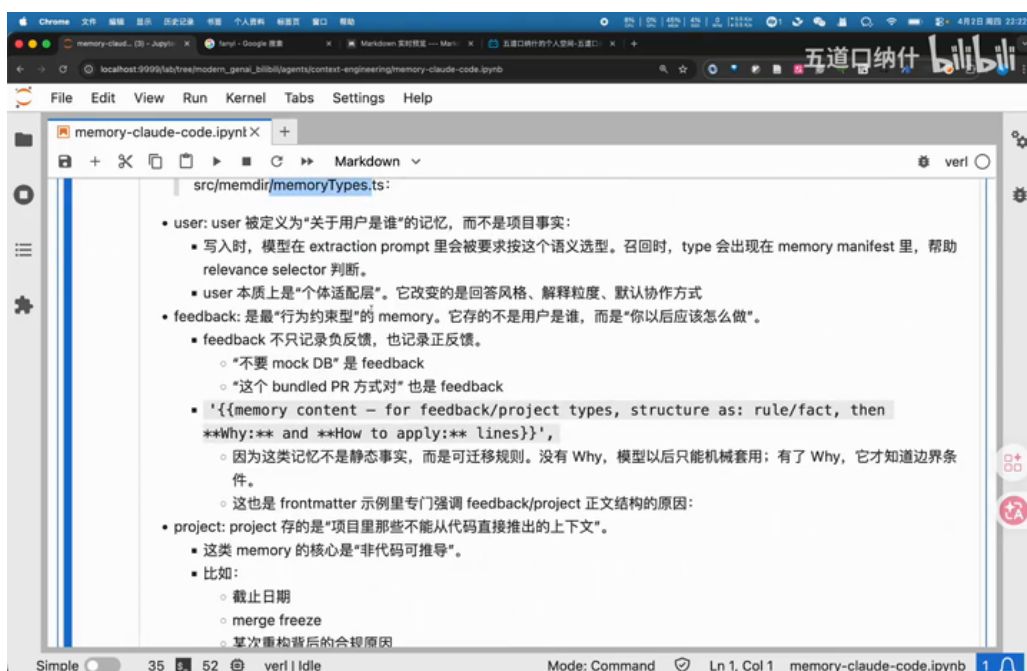


图 1: 视频中对四种记忆类型的讲解画面¹

2.5 本章小结

四类记忆的概括

- **User:** 回答了这 个用户是谁
- **Feedback:** 我 (Coding Agent) 以后该怎么做
- **Project:** 这 个项目当前处在一个什么样的外部语境里
- **Reference:** 如 果要查外部信息，应该去哪找”0308itemize

¹视频画面时间区间：00:07:00-00:08:30。

3 Manifest：声明式元数据文件

视频引入了 **Manifest** 的概念——它是对 Memory 目录中已有记忆文件的一个轻量级目录摘要，不是全文内容，也不是索引文件。

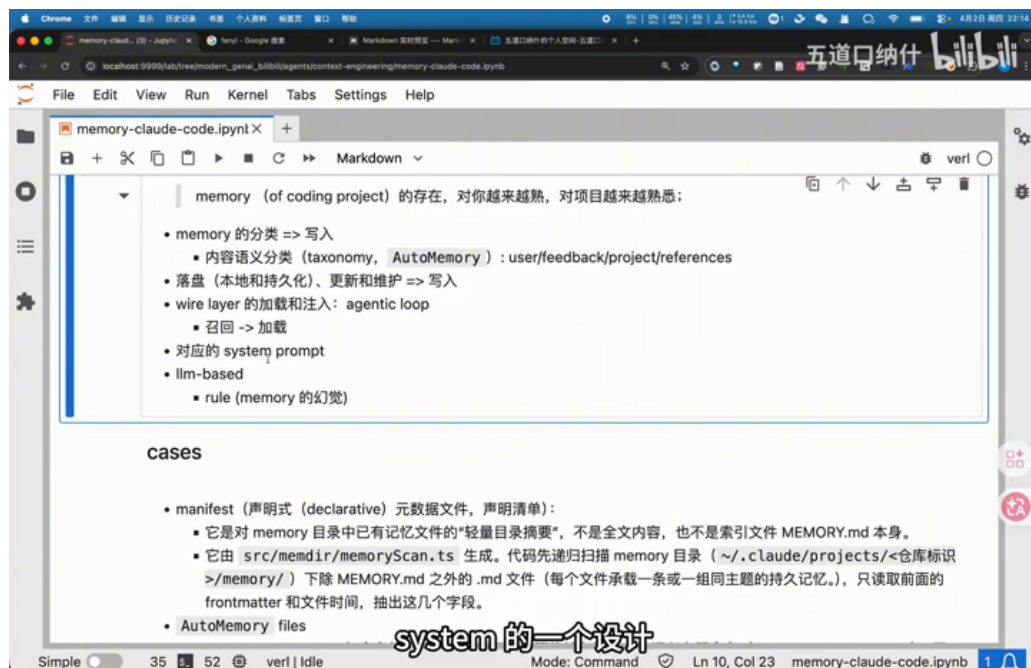


图 2: 视频中关于 Manifest 系统的讲解画面²

3.1 MEMORY.md：索引文件

MEMORY.md 是索引文件——类似于在发送第一条 User Query 之前，系统注入的 Skill 索引。它包含每个记忆文件的名称和简短描述。

3.2 目录结构

```
1 ~/.claude/projects/<project-id>/memory/  
2 +-- MEMORY.md # 索引文件  
3 +-- user_role.md # user 类型  
4 +-- feedback_terse.md # feedback 类型  
5 +-- project_freeze.md # project 类型  
6 +-- reference_linear.md # reference 类型
```

Listing 2: Auto Memory 目录结构

²视频画面时间区间：00:03:00-00:04:30。

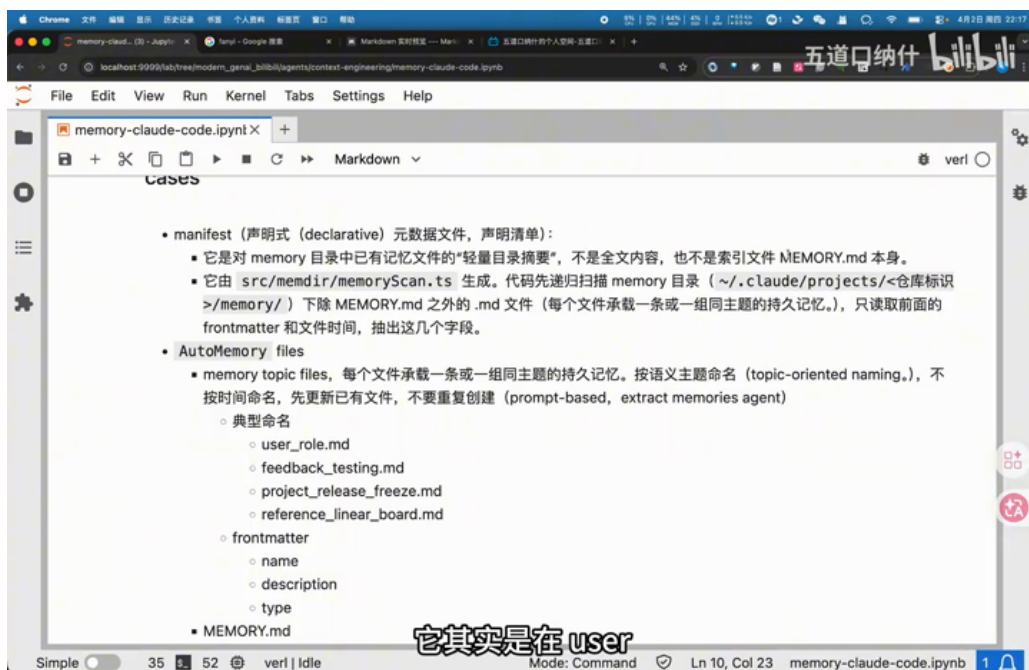


图 3: 视频中展示的 Auto Memory 目录结构³

3.3 Front Matter 格式

每个记忆文件使用 YAML Front Matter:

```

1 ---
2 name: {记忆名}
3 description: {一行描述——用于相关性召回，要足够具体}
4 type: {user | feedback | project | reference}
5 ---
6 {正文内容——feedback/project 类型需含 Why 和 How to apply}

```

Listing 3: 记忆文件的 Front Matter 格式

视频强调：分类按**语义主题**命名，不是按时间命名。写入时先更新已有文件，不重复创建。

4 写入管线：Extract Memories Agent

记忆的自动写入是视频的重点之一。

4.1 写入时机

视频中关于写入时机的讲解（00:14:00–00:15:00）

不是主模型直接写，而是在这一轮会话结束之后，fork 一个 Agent 去做总结和提取。这个概念叫 Turn——用户有一个 Query，Coding Agent 进入多轮工具调用，形成最终回答。Turn 结束之后会 fork 一个 Agent 去做总结和提取。

³视频画面时间区间：00:05:00–00:06:30。

三步流程：

1. 扫描已有记忆，给 Extract Agent 一个 **Manifest**（清单）
2. Forked Agent 自己决定——更新旧文件还是写新文件
3. 限制权限，只允许在 Memory 目录内操作

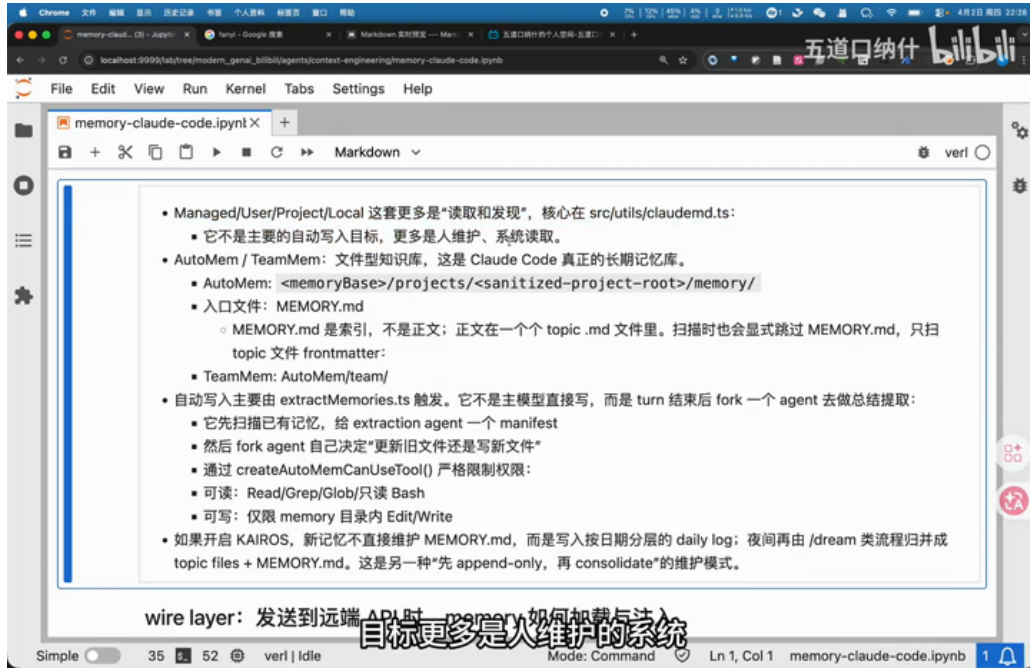


图 4: 视频中关于 Extract Memories Agent 的讲解画面⁴

4.2 关键约束

- 互斥：若主 Agent 已写入记忆，Extract Agent 跳过本轮
- 工具权限沙箱：只读 Bash + 仅限 Memory 目录写入
- 硬上限：最多 5 轮对话，防止验证兔子洞
- 只提取不验证：禁止 grep/read code，避免陷入代码验证循环

5 召回与注入：Wire Layer

5.1 召回流程

视频详细介绍了从 Memory 文件中召回最相关内容的过程：

1. 扫描 Memory 目录，读取每个.md 文件的 Front Matter（前 30 行）
2. 格式化为 Manifest：< 文件名 > (< 时间戳 >)：< 描述 >

⁴视频画面时间区间：00:13:00-00:15:00。

3. 基于**名称和描述**判断相关性，最多选 5 个文件
4. 排除已展示的记忆，避免浪费召回槽位

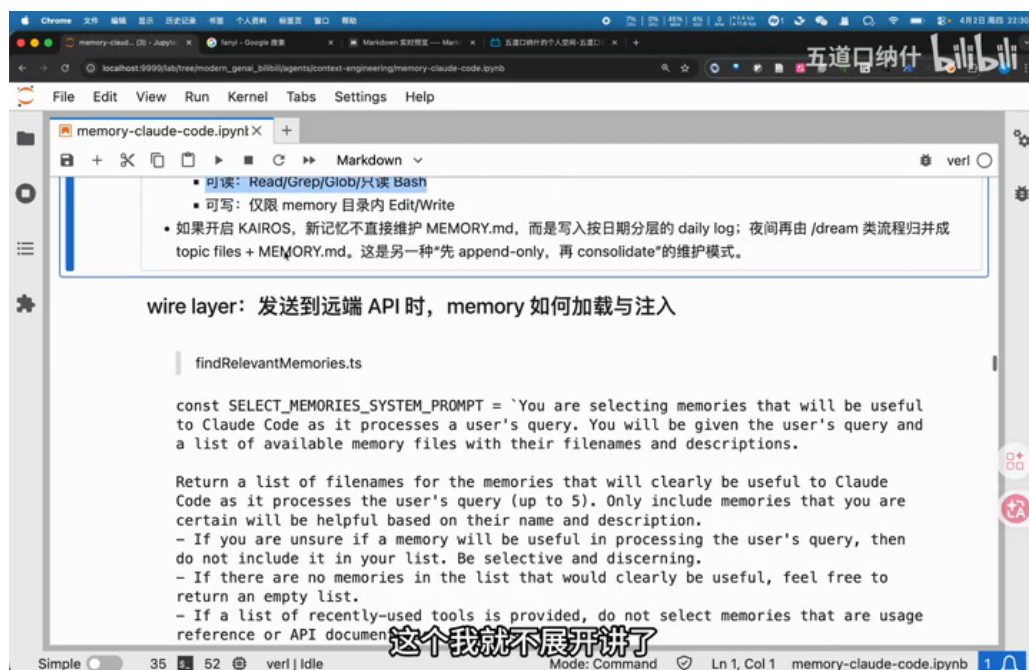


图 5: 视频中关于 Select Memory 的讲解画面⁵

5.2 Select Memory 系统提示词

视频解读了 Select Memory 的 System Prompt 的关键内容:

- **硬限制**: 最多发 5 个文件
- **判别力要求**: 如果不确定是否有用, 就不要包含
- **去重机制**: 如果当前对话已经在某个工具, 就不要召回该工具的相关记忆——因为工具的 Schema、Call、Results 已经在上下文中了

5.3 Memory 不是 Source of Truth

这是视频中最重要的一個警告:

Memory 幻觉防范

Memory 中如果提到某个具体符号 (文件、函数、Flag), 它只能被当做**历史的线索**, 而不能被当做**当前的事实**。

——因为 Memory 记录的是写下这条记忆时的认知, 而代码库是持续变化的。这个函数或文件可能在后来被重命名、删除、或者从未被真正合并到当前分支。

⁵ 视频画面时间区间: 00:15:00–00:16:30。

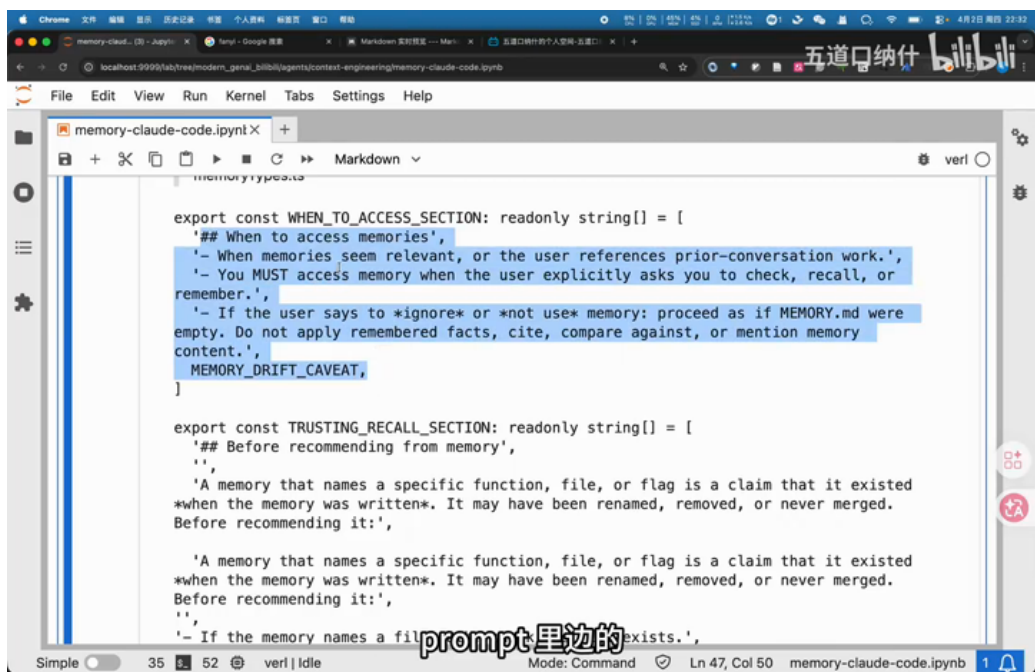


图 6: 视频中强调 Memory 不是 Source of Truth 的画面⁶

模型的要求

Prompt 不是在建议，而是在明确要求：

Memory 不是 Source of Truth。Memory 只是一个**历史的线索**。真正要给用户建议之前，必须回到当前代码、当前仓库中去验证。

5.4 注入方式

视频介绍了记忆注入 Agent Loop 的多种方式：

注入位置	内容	特点
System Prompt	Memory 策略——何时获取、如何使用	规则型，非事实
User Context	CLAUDE.md 内容	会话间变化
附件型 User Message	Relevant Memories	在真实 Query 前注入
Nested Memory	相关 CLAUDE.md / Rules	格式化文件

表 2: 记忆注入 Agent Loop 的四种方式

⁶视频画面时间区间：00:17:00–00:19:00。

6 总结与延伸

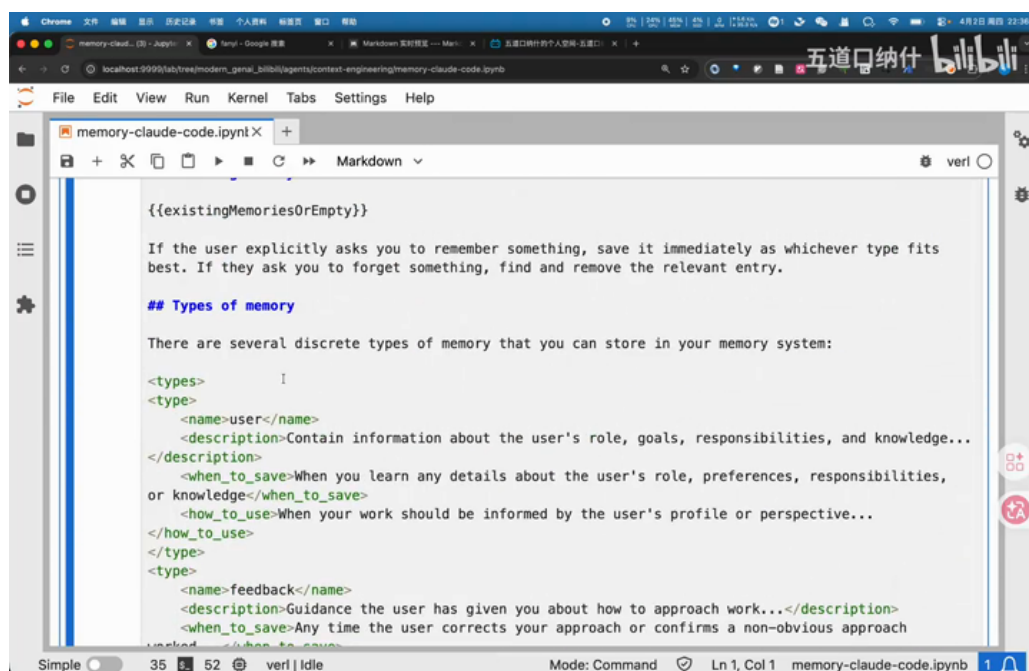


图 7: 视频总结部分画面 (00:20:00–00:21:37) ⁷

6.1 视频的核心理念

视频结尾总结 (00:20:30–00:21:37)

以上就是本期的全部内容。希望我们这一系列关于 Claude Code 的分析——第 13 期、14 期、15 期、16 期、17 期——大家可以带着这些线索、这些主题去读，比公认的更好的 Coding Agent——Claude Code 的源码。

6.2 关键设计原则

1. **Markdown 文件系统作为持久化层**——纯文本、可审查、可编辑、可版本控制
2. **写入时语义分类**——四类闭合分类在写入时就完成了粗过滤
3. **语言模型驱动**——从分类、写入到召回，每一步都是 Prompt-Based
4. **互斥协议**——三路写入管线通过互斥避免冲突
5. **验证而非盲信**——Memory 只是历史线索，必须回到当前代码验证

⁷ 视频画面时间区间：00:20:00–00:21:37。

6.3 关键数字汇总

指标	数值
记忆类型	4 种（闭合分类）
召回上限	5 条/次
Extract Agent 最大回合	5 轮
Session Memory 首次触发	10K tokens
视频时长	21:38

表 3: Claude Code 记忆系统的关键数字

6.4 拓展阅读

相关资源

- 视 频 源 码: https://github.com/wdkns/modern_genai_bilibili/blob/main/agents/context-engineering/memory-claude-code.ipynb
- Claude Code Memory 源码分析: <https://zhuanlan.zhihu.com/p/2026432792104280269>
- Agent-Memory 架构拆解: <https://zhuanlan.zhihu.com/p/2032173784879330093>
- how-claude-code-works 记忆系统文档: <https://github.com/Windy3f3f3f3f/how-claude-code-works/blob/main/docs/08-memory-system.md>